

Project Report: Numerical Simulation of Vibrations in a Stretched String

Comprehensive Analysis and Implementation (Questions 1 & 2)

Student Submission

June 21, 2026

Introduction

In this project, I am modeling the physical behavior of a vibrating string of length L that is securely fastened at both of its ends. The movement of this string is governed by the one-dimensional forced wave equation:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2} + f(x, t), \quad \text{for } 0 < x < L \text{ and } t > 0 \quad (1)$$

where $u(x, t)$ represents the vertical displacement of the string at position x and time t . The parameter $c = \sqrt{T/\rho}$ is the wave speed (dependent on tension T and mass density ρ), and $f(x, t)$ acts as an external driving force.

Because the string is physically tied down at the ends, I must enforce the Dirichlet boundary conditions:

$$u(0, t) = 0 \quad \text{and} \quad u(L, t) = 0 \quad \text{for } t > 0 \quad (2)$$

My objective in this first phase of the project is to discretize this continuous mathematical model into an implicit finite difference scheme and translate it into an efficient algorithmic structure that I can implement in Python.

—

1 Finite Difference Discretization (Question 1)

To computationally solve the wave equation, I first need to transition from continuous space and time into a discrete grid.

I defined a uniform spatial mesh by dividing the string into N segments, giving a step size of $\Delta x = \frac{L}{N}$. The grid points are located at $x_i = i\Delta x$ for $i = 0, 1, \dots, N$. Similarly, I discretized time using a step size Δt , resulting in time levels $t^n = n\Delta t$. I use the notation u_i^n to represent my numerical approximation of the exact displacement $u(x_i, t^n)$.

Approximating the Derivatives

To construct the numerical scheme, I replaced the continuous partial derivatives with centered finite difference approximations, which are derived from Taylor series expansions.

For the **time derivative** at the current state (x_i, t^n) , I used the standard second-order centered formula:

$$\frac{\partial^2 u}{\partial t^2}(x_i, t^n) \approx \frac{u_i^{n+1} - 2u_i^n + u_i^{n-1}}{\Delta t^2} \quad (3)$$

For the **spatial derivative**, I had a choice between an explicit or implicit formulation. As required by the project, I opted for the *implicit* scheme. This means I evaluated the spatial derivative at the *future* time step t^{n+1} . While this makes the algebra more complex, it guarantees absolute numerical stability, allowing me to choose larger time steps Δt without the simulation "blowing up."

$$\frac{\partial^2 u}{\partial x^2}(x_i, t^{n+1}) \approx \frac{u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}}{\Delta x^2} \quad (4)$$

Constructing the Discrete Scheme

Substituting these approximations back into the governing wave equation, and evaluating the external force $f(x, t)$ at the known current state (x_i, t^n) , I arrived at the implicit discrete equation:

$$\frac{u_i^{n+1} - 2u_i^n + u_i^{n-1}}{\Delta t^2} = c^2 \frac{u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}}{\Delta x^2} + f(x_i, t^n) \quad (5)$$

—

2 Matrix Formulation of the Implicit Scheme (Question 2)

Because the implicit scheme references three unknown future values ($u_{i-1}^{n+1}, u_i^{n+1}, u_{i+1}^{n+1}$), I cannot simply solve for u_i^{n+1} in isolation. Instead, I need to solve a system of simultaneous linear equations.

Algebraic Rearrangement

My first step was to isolate all the future terms on one side of the equation. I multiplied everything by Δt^2 :

$$u_i^{n+1} - 2u_i^n + u_i^{n-1} = \left(\frac{c^2 \Delta t^2}{\Delta x^2} \right) (u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}) + \Delta t^2 f(x_i, t^n) \quad (6)$$

To keep the equations clean, I defined the dimensionless constant $r = \frac{c^2 \Delta t^2}{\Delta x^2}$. Substituting r gives:

$$u_i^{n+1} - 2u_i^n + u_i^{n-1} = r (u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}) + \Delta t^2 f(x_i, t^n) \quad (7)$$

I then shifted all $n + 1$ terms to the left side and kept the known n and $n - 1$ terms on the right side:

$$u_i^{n+1} - ru_{i+1}^{n+1} + 2ru_i^{n+1} - ru_{i-1}^{n+1} = 2u_i^n - u_i^{n-1} + \Delta t^2 f(x_i, t^n) \quad (8)$$

By factoring out u_i^{n+1} and ordering the indices spatially, I obtained the generalized nodal equation for any interior point ($i = 1, 2, \dots, N - 1$):

$$-ru_{i-1}^{n+1} + (1 + 2r)u_i^{n+1} - ru_{i+1}^{n+1} = 2u_i^n - u_i^{n-1} + \Delta t^2 f(x_i, t^n) \quad (9)$$

Applying Boundary Conditions and Building Matrix A

To build the matrix system $Ax = b$, I analyzed the interactions at the boundaries. The string is fixed, so I know definitively that $u_0^{n+1} = 0$ and $u_N^{n+1} = 0$.

When calculating the displacement for the first internal node ($i = 1$), the equation is:

$$-ru_0^{n+1} + (1 + 2r)u_1^{n+1} - ru_2^{n+1} = 2u_1^n - u_1^{n-1} + \Delta t^2 f(x_1, t^n) \quad (10)$$

Because $u_0^{n+1} = 0$, that specific term drops out entirely. The same logic applies at the other end ($i = N - 1$), where the u_N^{n+1} term drops out.

By stacking these equations for all interior points, I can express the system as a matrix equation $AU^{n+1} = 2U^n - U^{n-1} + \Delta t^2 F^n$, where U and F are column vectors of length $N - 1$, and my matrix A takes on a perfectly tridiagonal structure:

$$A = \begin{pmatrix} 1+2r & -r & 0 & \dots & 0 \\ -r & 1+2r & -r & \dots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \dots & -r & 1+2r & -r \\ 0 & \dots & 0 & -r & 1+2r \end{pmatrix} \quad (11)$$

3 Algorithmic Implementation and Coding Strategy

When moving to the Python implementation, my primary concern was algorithm efficiency. The simulation requires solving the linear system $AU^{n+1} = b$ at *every single time step*.

Initially, I considered using a standard solver like `numpy.linalg.solve()` inside the time loop. However, I realized that because the matrix A is constant (it depends only on the fixed parameters $c, \Delta x, \Delta t$), recompiling or factoring it at every time step is highly inefficient.

Instead, I implemented an LU decomposition strategy. Using `scipy.linalg.lu`, I factored A into its lower and upper triangular components ($A = PLU$) exactly once before the simulation loop begins. Inside the loop, I just perform forward and backward substitution. This simple architectural decision reduces the computational complexity of each time step significantly, allowing for high-resolution meshes without performance bottlenecks.

```

1 import numpy as np
2 import scipy.linalg as la
3
4 # =====
5 # 1. Parameter Initialization
6 # =====
7 L = 1.0          # Length of the string
8 T_end = 2.0      # Total simulation time
9 N = 50           # Number of spatial intervals
10 M = 100          # Number of time steps
11 c = 1.0          # Wave speed
12
13 # Calculate step sizes and the dimensionless ratio 'r'
14 dx = L / N
15 dt = T_end / M

```

```

16 r = (c * dt / dx)**2
17
18 # Generate spatial and temporal grids
19 x = np.linspace(0, L, N+1)
20 t = np.linspace(0, T_end, M+1)
21
22 # Initialize the solution matrix U (rows = space, columns = time)
23 U = np.zeros((N+1, M+1))
24
25 # Define the external forcing function (given in the project brief)
26 def f(x, t):
27     return -3 * np.pi**2 * np.sin(np.pi * x) * np.cos(2 * np.pi * t)
28
29 # =====
30 # 2. Applying Initial Conditions
31 # =====
32 # Initial displacement u(x,0)
33 U[:, 0] = np.sin(np.pi * x)
34
35 # To find the first time step, I apply the initial velocity condition u_t(
    x,0) = 0.
36 # Using a first-order forward difference: (u[:,1] - u[:,0])/dt = 0
37 U[:, 1] = np.copy(U[:, 0])
38
39 # =====
40 # 3. Matrix Construction & LU Decomposition
41 # =====
42 # Build the tridiagonal matrix A
43 main_diag = (1 + 2*r) * np.ones(N-1)
44 off_diag = -r * np.ones(N-2)
45 A = np.diag(main_diag) + np.diag(off_diag, k=1) + np.diag(off_diag, k=-1)
46
47 # OPTIMIZATION: Compute LU decomposition once, before the loop starts
48 P, L_mat, U_mat = la.lu(A)
49
50 # =====
51 # 4. Main Simulation Loop
52 # =====
53 # Loop over time, starting from n=1 (since n=0 and n=1 are already set)
54 for n in range(1, M):
55
56     # Extract interior nodes from current and previous states
57     U_current = U[1:N, n]

```

```

58     U_prev = U[1:N, n-1]
59
60     # Evaluate the force vector at the current time step
61     F_n = f(x[1:N], t[n])
62
63     # Construct the known right-hand side vector 'b'
64     b = 2 * U_current - U_prev + (dt**2) * F_n
65
66     # Solve the system efficiently using the pre-computed LU matrices
67     # Step 1: Forward substitution (L * y = P.T * b)
68     y = la.solve_triangular(L_mat, np.dot(P.T, b), lower=True)
69
70     # Step 2: Backward substitution (U * U_next = y)
71     U_next = la.solve_triangular(U_mat, y, lower=False)
72
73     # Store the results. Boundary nodes naturally remain 0.0.
74     U[1:N, n+1] = U_next

```

Listing 1: Optimized Implicit Wave Equation Solver